Smart Support Ticket Classifier

End-to-end NLP pipeline · 6 models · Flask API · Next.js demo

2,229	610	962	0.74
Raw Tickets	Duplicates Found	Clean Samples	Best Macro F1

- Gaurang Patil

The Problem

IT helpdesks receive hundreds of tickets daily. Every ticket needs to be manually read and routed to the right team — a slow, repetitive, error-prone process.

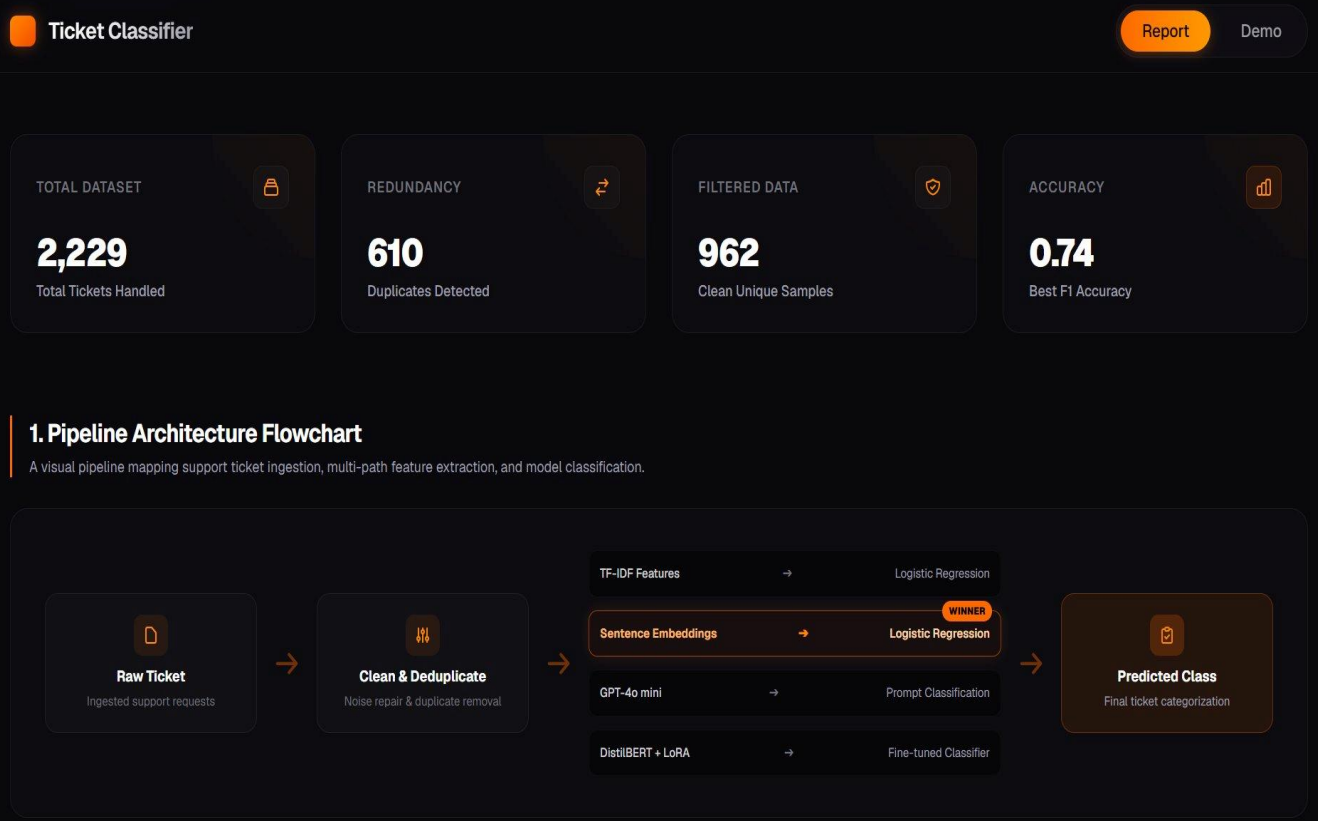
6 Target Categories:

- Fileservice
- Support general
- Software
- O365
- Active Directory
- Computer-Services

Our Solution

Automatically classify incoming tickets using ML models — eliminating manual triage and routing instantly to the correct team.

Dataset & Pipeline Architecture

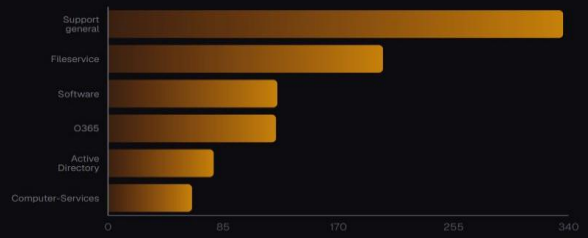


Data Audit & Cleaning

2. Dataset & Class Distribution

Comprehensive analysis of ticket categories and language frequencies across the entire cleaned dataset.

Class Label Distribution



Ticket Language Distribution

English	721	(77.0%)
Portuguese	94	(10.0%)
German	60	(6.4%)
Unknown	50	(5.3%)
Spanish	11	(1.2%)

3. Data Preprocessing & Validation

Rigorous data sanitization protocols implemented to group duplicates, scrub label noise, and isolate testing baselines.

01 Deduplication

The training set dataset size was optimized and shrank cleanly from 1572 down to 962 samples by stripping redundant requests.

02 EOL Class Removal

Scrubbed the End-of-Life category because investigation revealed 44 out of 45 submitted samples were identical automated templates.

03 Label Cleaning

Resolved structural label noise where instances of identical ticket text were submitted under two entirely separate classifications.

04 Train/Test Preservation

Guaranteed model audit validity by holding the evaluation test dataset completely untouched throughout the preprocessing pipeline.

6 Models Compared

TF-IDF + LR — **F1: 0.69**

Bag-of-words baseline

Embeddings + LR ⭐ — **F1: 0.74**

Multilingual semantic model

Embeddings + Synthetic — **F1: 0.72**

Augmented training data

GPT-4o mini Zero-shot — **F1: 0.64**

No training, just prompting

GPT-4o mini Few-shot — **F1: 0.71**

6 examples in prompt

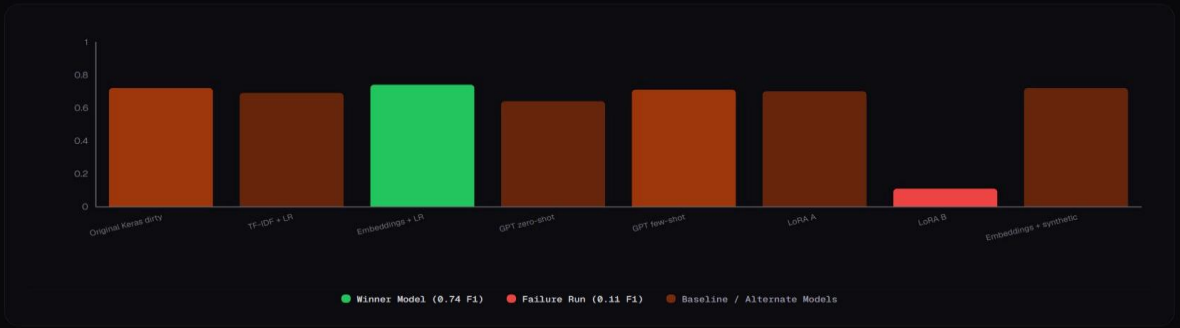
DistilBERT + LoRA — **F1: 0.70**

Fine-tuned transformer

Model Performance Results

4. Comparative Model Performance (Macro F1)

Evaluation metrics across all eight model architectures, highlighting our optimal sentence embeddings approach.



5. Performance Metrics Reference Table

Detailed breakdown of F1-scores, accuracy rates, real-world ticket testing, custom evaluations, and core insights.

MODEL	MACRO F1	ACCURACY	REAL TICKETS	CUSTOM 50	CORE TAKEAWAY
Original Keras (dirty)	0.72	0.79	–	–	Inflated by dirty data
TF-IDF + LR	0.69	0.75	–	–	Honest baseline
• Embeddings + LR	**0.74**	**0.78**	**0.83**	**0.59**	Best overall
GPT zero-shot	0.64	0.67	0.71	0.80	Flexible, not robust
GPT few-shot	0.71	0.72	0.68	0.86	Strong on clean prompts
LoRA A	0.70	0.78	0.69	0.48	Needs more data
LoRA B	0.11	0.30	–	–	Failure run
Embeddings + synthetic	0.72	0.78	0.73	0.84	Better external, weaker real

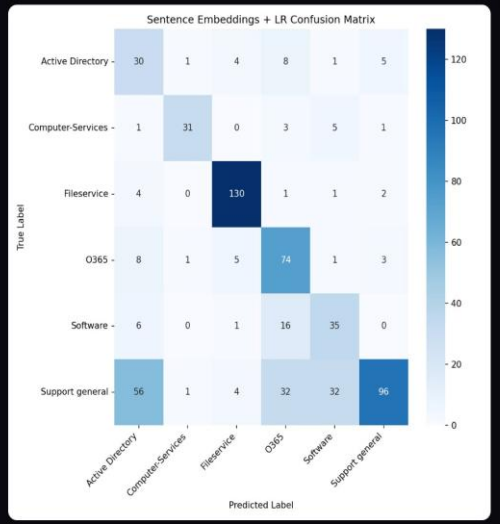
Confusion Matrices — Best vs GPT Zero-shot

Sentence Embeddings + LR (Winner)

6. Model Confusion Matrix Gallery

Class-by-class confusion matrix selector to audit misclassifications, class recall rates, and model errors.

Embeddings + LR | TF-IDF + LR | Embeddings + Synthetic | GPT Zero-shot | GPT Few-shot | LoRA Model A | LoRA Model B

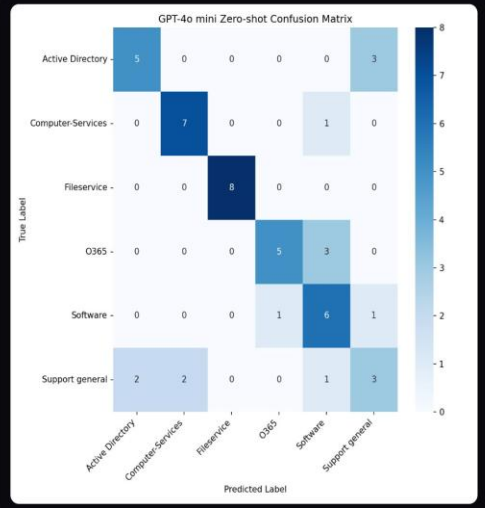


GPT-4o mini Zero-shot

6. Model Confusion Matrix Gallery

Class-by-class confusion matrix selector to audit misclassifications, class recall rates, and model errors.

Embeddings + LR | TF-IDF + LR | Embeddings + Synthetic | GPT Zero-shot | GPT Few-shot | LoRA Model A | LoRA Model B



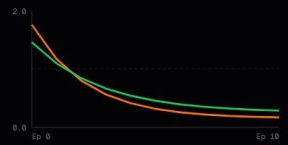
LoRA Fine-tuning — Success vs Failure

7. LoRA Fine-Tuning Loss Curves

Side-by-side comparison of training loss convergence (Model A) versus gradient divergence and failure (Model B).

[LOSS CURVES MOCKUP] • Loss Converged

LoRA Model A Training History



Training Loss — Validation Loss

[LOSS CURVES MOCKUP] • Loss Diverged

LoRA Model B Training History

Training Loss — Validation Loss

8. Executive Insights & Lessons Learned

Four foundational learnings established through auditing neural representations, data quality, and LLM constraints.

Dirty Data Inflated Metrics

Automated templates artificially inflated early evaluation performance. For example, EOL obtained an flawless 1.00 F1 score solely due to 44 identical templates.

Embeddings Beat TF-IDF

Continuous semantic representations outperformed standard bag-of-words pipelines, boosting F1 classification scores by +0.05 overall, and +0.09 on real tickets.

GPT Wins on Clean Prompts Only

Large language models demonstrated extreme prompt dependency, scoring 0.86 F1 on carefully curated custom test sets, but dropping to 0.68 F1 on raw real tickets.

Synthetic Data Shifted Optimization

Synthetically augmented samples skewed the boundary optimization of the embedding vectors, causing real-ticket evaluation scores to decay from 0.83 down to 0.73 F1.

Key Findings

Dirty Data Inflated Metrics

EOL class got 1.00 F1 on 44 identical templates. The data audit was the most important step.

Embeddings Beat TF-IDF

+0.05 macro F1 and +0.09 on real tickets. Semantic meaning matters more than word frequency.

GPT Wins on Clean Text Only

0.86 on custom set but dropped to 0.68 on real helpdesk traffic. Prompt design doesn't fix distribution shift.

Synthetic Data Shifted Optimization

Augmentation boosted external eval but real-ticket F1 dropped 0.83 → 0.73. LLM data is too clean.

Winner: Sentence Embeddings + LR

0.74	0.78	0.83
Macro F1	Accuracy	Real Tickets

Why it won:

- Handled short, noisy, multilingual tickets better than bag-of-words
- More robust on real-world data than GPT (0.83 vs 0.68)
- No API cost, no prompt design fragility, fast inference
- Simple to explain, defend, and iterate on

Next Steps

- Augment with real helpdesk logs
- Retry LoRA on larger clean corpus
- Add confidence calibration layer
- Deploy with Docker + K8s

Built with: scikit-learn · sentence-transformers · PEFT · OpenAI · Flask · Next.js